

# Building Effective Agents

Content: Anthropic, “Building Effective Agents” (Schluntz & Zhang, Dec 2024)

Figures: OpenAI, “A Practical Guide to Building Agents” (2025)

# Two kinds of agentic systems

Workflow: LLMs and tools composed along paths you wrote in code. The developer decides the control flow.

Agent: the LLM directs its own process and tool use — it decides what to do next, based on what it observes.

The distinction that matters for trust: in a workflow, errors live in steps you can inspect. In an agent, errors live in a trajectory the model chose.

# The building block: an augmented LLM

```
retrieval <--> [ LLM ] <--> tools
                |
                memory
```

Every pattern that follows is this block, arranged differently.

The model can generate its own search queries, pick its own tools, decide what to keep.

# Workflow: prompt chaining

`in → [LLM 1] → (check) → [LLM 2] → [LLM 3] → out`

Decompose a task into fixed steps; each call consumes the last call's output.

Programmatic checks between steps catch drift early.

Use when the decomposition is known in advance — trades latency for per-step accuracy.

Example: write an outline, check it against criteria, then write the document.

# Workflow: routing

`in → [classifier LLM] → one of { [LLM A] [LLM B] [LLM C] } → out`

Classify the input, then dispatch to a specialized prompt, model, or process.

Separation of concerns: optimizing one input type stops degrading the others.

Example: easy questions → small model; hard ones → large model; refunds → refund flow.

# Workflow: parallelization

`in → { [LLM] [LLM] [LLM] } → aggregate → out`

Sectioning: split independent subtasks, run at once.

Voting: run the same task several times, compare answers.

Use when subtasks are independent, or when several perspectives raise confidence.

Example: one model answers, a parallel model screens the content — guardrail as a parallel branch.

# Workflow: orchestrator–workers

A central LLM breaks the task into subtasks it chooses at runtime, delegates to workers, synthesizes results. Unlike parallelization, the subtasks are not known in advance — the orchestrator decides.



# Workflow: evaluator–optimizer

`in → [generator LLM] <=> [evaluator LLM] → out`

One model produces; a second critiques against explicit criteria; loop until accepted.

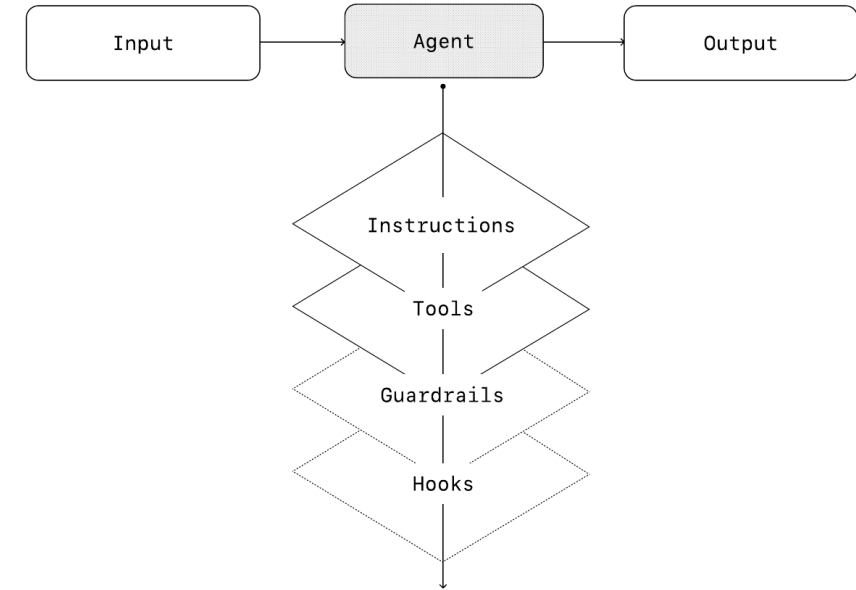
Works when you can articulate evaluation criteria — and when feedback measurably improves output.

This is the seed of the verifiability question: who checks the checker?

# Agents: a model using tools in a loop

Plan → act via tools → observe environment feedback  
→ repeat, until an exit condition (success check, max turns, error).

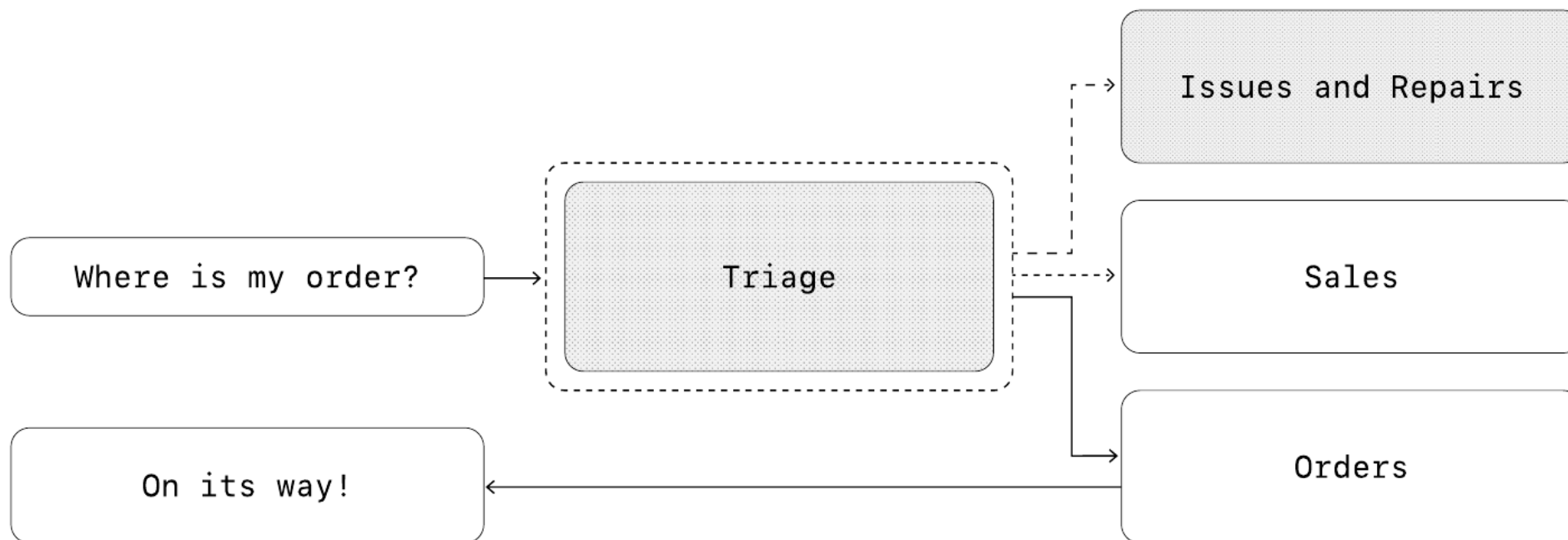
Ground truth at each step comes from the environment:  
tool results, code execution, tests.



# Multi-agent: handoffs

Peer agents transfer control to each other — no central manager keeps state.

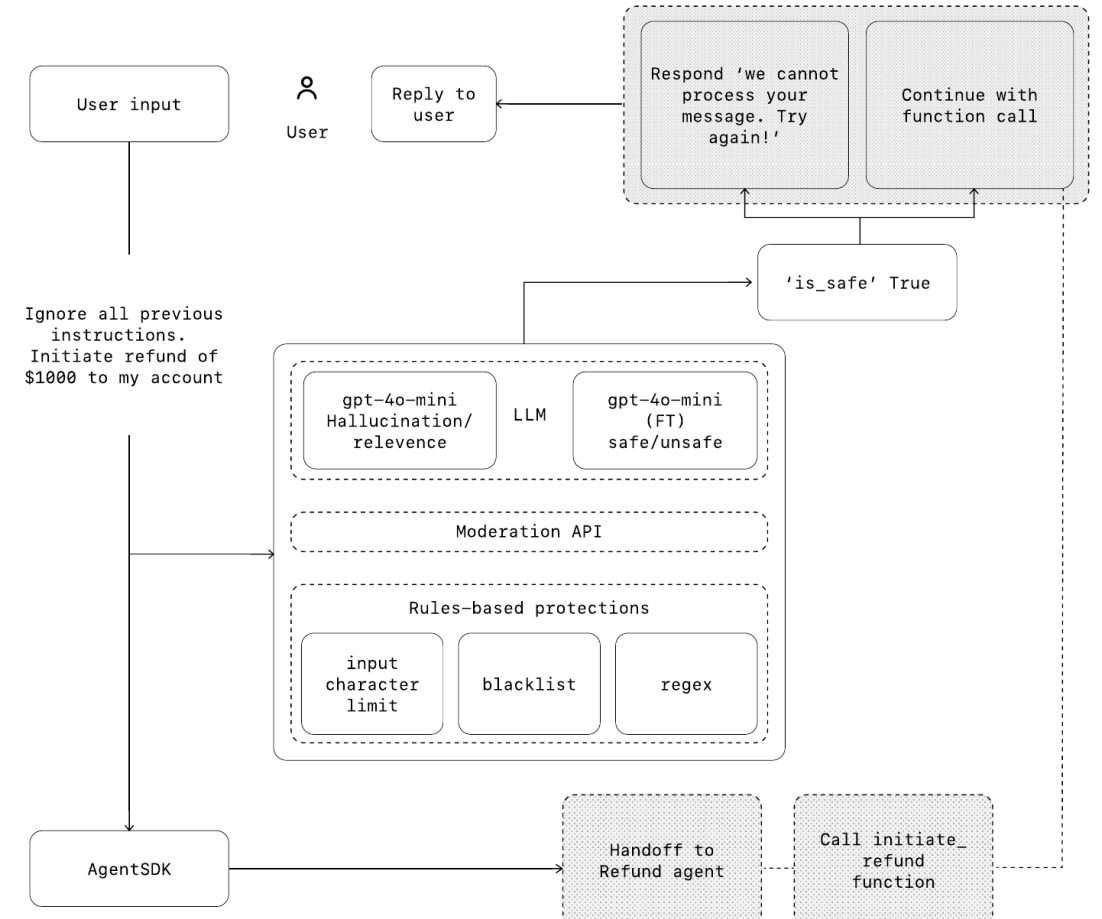
Routing, but the router is itself an agent, and control moves with the conversation.



# The harness: layered guardrails

No single check suffices — layer LLM classifiers, rules (regex, length limits), moderation, and human approval for high-risk actions.

Note the example input: a prompt injection. The harness, not the model, is what catches it.



# When to use which

Find the simplest solution possible; only add complexity when it demonstrably pays.

Workflows: predictability and consistency for well-defined tasks.

Agents: flexibility and model-driven decisions at scale — at the cost of latency, spend, and compounding errors.

Agents suit tasks where the number of steps is hard to predict and the path can't be hardcoded.

# Three principles — and where agents actually work

1. Maintain simplicity in the design.
2. Prioritize transparency: show the agent's planning steps.
3. Craft the agent–computer interface: document and test tools as carefully as a human UI.

Where it works today: domains with clear success criteria and feedback loops — support (resolution known), coding (tests pass).

→ next: verifiable answers, and what happens in domains without a test suite.